Linux Workstation Bring-Up Guide

This guide is tailored for the following workstation build, with the goal of teaching a modern Linux bring-up flow while also producing a stable, validated Ubuntu workstation suitable for graphics, display, robotics, and systems experimentation.

Target hardware

- CPU: AMD Ryzen 7 7800X3D
- Cooler: Corsair iCUE LINK Titan 240 RX
- Motherboard: ASUS ProArt B850-CREATOR WiFi NEO (AM5, ATX)
- RAM: Kingston Fury Beast 64 GB DDR5-6000
- SSD: Samsung 990 Pro 2 TB NVMe
- GPU: ASUS Prime GeForce RTX 5070
- PSU: Corsair RM750x ATX 3.1
- Case: Corsair 4000D RS ARGB Frame
- Displays: 2x ASUS ROG Strix 32-inch 4K OLED 240 Hz
- OS target: Ubuntu 24.04 LTS

Goals

- Refresh the full PC bring-up workflow end-to-end.
- Establish a stable Linux baseline before doing higher-risk experiments.
- Learn how to validate memory, thermals, storage, GPU, suspend/resume, and display behavior.
- Build confidence that the machine is reliable enough for longer development sessions and future low-level work.

Bring-up strategy

Use a staged approach:

1. Build and POST successfully with minimal variables.
2. Update firmware and configure BIOS conservatively.
3. Install Ubuntu in a simple, known-good configuration.
4. Install the current NVIDIA open driver stack appropriate for RTX 5070 on Ubuntu 24.04. [cite:1][cite:2]
5. Add monitors, higher refresh rates, stress tests, and development tools only after the base platform is stable.

6. Keep notes as you go so you can correlate any later issue with a configuration change.

1. Physical assembly

Assembly checklist

Before applying power:

- Install the CPU carefully and verify the AM5 socket cover and retention arm behave normally.
- Mount the AIO according to Corsair guidance, preferably with tubes down if front-mounted, or radiator above pump height if top-mounted, so air is less likely to collect in the pump.
- Install RAM in the recommended dual-channel slots from the motherboard manual, usually A2 and B2.
- Install the Samsung 990 Pro into the primary CPU-attached M.2 slot.
- Seat the GPU fully in the primary PCIe x16 slot and fully latch it.
- Connect GPU power exactly as specified by the ASUS/Corsair cabling guidance; avoid partially seated high-power connectors.
- Connect CPU EPS power, 24-pin ATX power, front panel leads, USB headers, and radiator fans/pump headers.
- For the first boot, connect only one monitor, one keyboard, and Ethernet if available.

First power-on expectations

On the first boot, AM5 may train memory and restart multiple times. Do not interrupt it unless it is clearly stuck for an unusually long time.

Expected good signs:

- Fans spin up.
- Motherboard status LEDs progress through CPU, DRAM, VGA, and BOOT.
- You eventually reach BIOS/UEFI setup.

Potential issues to catch early:

- DRAM LED never clears, which often means a seating or memory-training issue.
- VGA LED stays on, which can indicate GPU seating, power, or monitor cabling issues.
- No display at all, which may be caused by wrong output selection, poor cable choice, or an immature GPU driver once you get to Linux.[cite:1][cite:2]

Physical validation

Perform these checks before moving on:

1. Verify BIOS reports the full 64 GB of memory.
2. Verify the CPU is detected correctly.
3. Verify the NVMe SSD appears in storage devices.
4. Check that CPU temperature in BIOS is sensible at idle, not racing upward continuously.
5. Check that pump or AIO tach feedback is present if your board exposes it.

If idle BIOS temperatures rise rapidly past expected idle behavior, power down and inspect:

- Cooler mounting pressure.
- Thermal paste application.
- Pump power/header connection.
- Protective film removal from the cold plate.

2. BIOS update and baseline settings

BIOS update

Before installing Ubuntu, update the motherboard BIOS to the latest stable version available from ASUS. Firmware maturity matters on new AM5 boards for memory training, PCIe behavior, and general compatibility.

Typical procedure:

1. Download the latest BIOS for the exact motherboard model from ASUS support on another machine.
2. Copy the BIOS file to a FAT32-formatted USB stick.
3. Enter BIOS.
4. Use ASUS EZ Flash to select the BIOS file.
5. Do not interrupt power during the update.
6. After the update, load optimized defaults once.

Recommended BIOS settings

Start with conservative, stability-oriented settings:

- Boot mode: UEFI only.
- Disable CSM/legacy boot.
- Enable SVM/AMD-V if you want virtualization.
- Enable IOMMU for passthrough, VFIO experiments, and some advanced Linux workflows.

- Leave PCIe generation on Auto initially.
- Leave CPU tuning on stock settings initially.
- Leave PBO off or default initially.
- For storage, use AHCI for SATA devices.

Memory setup:

- Initially boot with JEDEC/default memory settings first.
- Once the system is stable, enable EXPO for DDR5-6000.
- If instability later appears under memory stress, reduce to DDR5-5600 before assuming hardware is defective.

Secure Boot:

- Disable it if you want the simplest NVIDIA driver path.
- Keep it enabled only if you intentionally want to learn module signing and MOK enrollment; Blackwell-era NVIDIA driver setup on Ubuntu may otherwise require extra signing steps with Secure Boot enabled.[cite:2]

BIOS validation

After applying settings, verify:

- BIOS still detects all hardware.
- Fan curves look sane.
- Boot order is correct.
- No unexpected warning screens appear on reboot.

3. Pre-OS memory validation with Memtest86+

This is one of the most useful early validation steps because it exercises memory before Linux, drivers, or filesystems complicate the picture.

What Memtest86+ is

Memtest86+ is a standalone memory test environment that boots directly and repeatedly writes patterns into RAM, reads them back, and checks for corruption. If memory settings, CPU memory controller behavior, or DIMM stability are marginal, Memtest86+ often finds it before the system fails in more subtle ways.

When to run it

Run it at two points:

1. **Before enabling EXPO**, at stock memory settings, to establish a baseline.
2. **After enabling EXPO**, to verify the higher-speed profile is actually stable.

How to run Memtest86+

There are two common ways:

Option A: Use the version bundled in Ubuntu/GRUB

After Ubuntu is installed, many systems can boot Memtest86+ from the GRUB menu.

1. Reboot the system.
2. Hold or tap the key that brings up GRUB if Ubuntu boots too quickly, often Shift or Esc depending on firmware behavior.
3. Look for a menu entry labeled something like "Memory test (memtest86+)".
4. Select it.
5. Let it run.

Option B: Use a dedicated bootable USB

This is more explicit and is useful if GRUB does not provide it.

1. Download the current Memtest86+ USB image from the official project site.
2. Write it to a USB drive using a tool like balenaEtcher, Rufus, or `dd`.
3. Boot from that USB in UEFI mode.
4. Start the test using the default settings first.

How long to run it

Practical guidance:

- Initial confidence check: 1 full pass.
- Better stability check after EXPO: 4 passes.
- High confidence / overnight validation: 8+ hours.

How to interpret results

Good result:

- Zero errors across the full run.

Bad result:

- Even one memory error is a problem worth investigating.

If errors occur:

1. Reseat both DIMMs.
2. Revert memory to stock/JEDEC settings.
3. Test one DIMM at a time in the recommended slot.

4. Test the other DIMM.
5. If each DIMM passes individually but fails together at EXPO, reduce memory speed or relax settings.
6. Check for BIOS updates if the platform is very new.

What Memtest86+ does not prove

A pass does **not** guarantee complete stability under every Linux workload, but it is a strong early filter. Some memory issues only appear later under mixed CPU, PCIe, and GPU load. That is why this guide includes in-OS memory and stress tests too.

4. Ubuntu installation

Installation approach

Use Ubuntu 24.04 LTS for a stable baseline. For this system, the safest path is usually to install Ubuntu first, update everything, and then install the NVIDIA driver after the first boot, rather than relying on whatever third-party graphics stack the installer happens to pull in.[cite:1] [cite:2]

Create the installer

1. Download the Ubuntu 24.04 LTS desktop ISO from Ubuntu.
2. Create a bootable USB in UEFI mode.
3. Disconnect any extra internal drives you do not want touched.

Install steps

1. Boot from the Ubuntu USB.
2. Choose the normal install path.
3. Use one monitor only.
4. If asked about proprietary drivers during install, prefer skipping that and handling NVIDIA after first boot.
5. Partition the drive manually if you want a predictable layout.

Suggested simple layout:

- EFI System Partition: 512 MB, FAT32.
- Root (/): remainder of the disk, ext4.

If you want snapshots and are willing to learn extra filesystem tooling, btrfs is also a valid option.

Initial post-install checks

After the first successful login:

1. Open a terminal.
2. Confirm the running kernel with `uname -r`.
3. Confirm the disk layout with `lsblk`.
4. Confirm that the root filesystem mounted as expected with `findmnt /`.
5. Check for obvious critical boot errors with:

```
sudo dmesg -l err,crit,alert,emerg
sudo journalctl -p err -b
```

Interpretation:

- A few benign warnings can happen, but repeated storage, GPU, or PCIe errors are not normal.
- Save the output if you see anything suspicious so you can compare later.

5. Base system update

Before touching graphics drivers, fully update Ubuntu.

Steps

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

After reboot:

```
uname -r
cat /etc/os-release
```

Validation

Confirm:

- The system reboots cleanly.
- Networking works.
- No new boot-critical errors appear in `journalctl -p err -b`.

This matters because newer kernels and packages often improve support for recent hardware.

6. NVIDIA driver bring-up for RTX 5070

RTX 5070 systems on Ubuntu 24.04 have commonly needed newer 57x-series or newer open NVIDIA drivers, rather than older defaults, to avoid no-display or incomplete support scenarios.[cite:1][cite:2]

Why use the open NVIDIA package path

Ubuntu packaging plus the graphics-drivers PPA is generally easier to maintain than the standalone `.run` installer, especially on a machine intended for long-term use.[cite:1][cite:2]

Installation steps

1. Add the graphics drivers PPA:

```
sudo add-apt-repository ppa:graphics-drivers/ppa
sudo apt update
```

2. See what Ubuntu recommends:

```
ubuntu-drivers devices
```

3. Install the recommended modern open driver. For example:

```
sudo apt install nvidia-driver-570-open
```

If Ubuntu offers a newer open package such as 580-open and it is the recommended path for your card, that is acceptable too.[cite:2]

4. Reboot.

If Secure Boot is enabled

You may be prompted to enroll a Machine Owner Key (MOK), or to sign the NVIDIA modules manually. That is expected behavior when proprietary or external kernel modules must load under Secure Boot.[cite:2]

High-level flow:

1. During driver installation, Ubuntu may ask you to set a temporary password for MOK enrollment.
2. On the next reboot, a blue MOK screen appears.
3. Choose to enroll the key.
4. Enter the password you set.
5. Reboot again.

If this is unfamiliar, the simplest path for a hobby workstation is to disable Secure Boot initially and revisit signed-module workflows later.

NVIDIA validation steps

After reboot, validate in layers.

Step 1: Confirm the hardware is visible

```
lspci | grep -i nvidia
```

Expected result:

- You should see the NVIDIA GPU listed on the PCI bus.

Step 2: Confirm the kernel modules loaded

```
lsmod | grep nvidia
```

Expected result:

- Several NVIDIA-related modules appear.

Step 3: Confirm the driver is working

```
nvidia-smi
```

Expected result:

- The RTX 5070 appears.
- A 57x or newer driver appears, depending on what you installed.[cite:1][cite:2]
- The command completes without an error about failed communication with the driver.

If `nvidia-smi` fails:

- Recheck Secure Boot state.
- Recheck whether the installed package is too old for the card.
- Recheck `journalctl -b | grep -i nvidia` for module load failures.

Step 4: Confirm OpenGL is hardware accelerated

Install tools if needed:

```
sudo apt install mesa-utils
```

Then run:

```
glxinfo | grep "OpenGL renderer"
```

Expected result:

- The renderer should identify NVIDIA hardware.
- It should **not** say `llvmpipe`, which would indicate software rendering.

Step 5: Confirm the displays are managed correctly

Install and open NVIDIA settings if desired:

```
sudo apt install nvidia-settings
nvidia-settings
```

Expected result:

- Connected monitors are visible.
- Resolution and refresh controls are available.

7. Display bring-up and dual-monitor validation

Because the monitors are 4K OLED panels capable of 240 Hz, avoid trying the most demanding display configuration first.

Recommended sequence

1. Start with one monitor on DisplayPort.
2. Set 3840x2160 at 60 Hz or 120 Hz.
3. Confirm stability.
4. Add the second monitor.
5. Move to 120 Hz on both.
6. Only then try 240 Hz if the cable, link training, and driver all behave properly.

Display validation steps

Resolution and refresh confirmation

Use the desktop display settings first.

Confirm for each monitor:

- Native 3840x2160 mode is available.
- Desired refresh rates are available.
- Orientation and placement are correct.

Command-line confirmation

For X11 sessions, run:

```
xrandr
```

Expected result:

- Both displays appear with the selected modes.

Motion and artifact check

Practical test:

1. Open several windows.
2. Drag them quickly between displays.
3. Open a browser and scroll rapidly.
4. Watch for flicker, blanking, dropped link, or random black flashes.

GPU-backed rendering check

Run a simple accelerated app or game and watch `nvidia-smi` in another terminal:

```
watch -n 1 nvidia-smi
```

Expected result:

- GPU processes appear while the app runs.

OLED-specific practical notes

For long Linux desktop sessions, reduce static UI exposure where convenient:

- Use dark theme.
- Auto-hide static panels if you prefer.
- Avoid leaving a terminal with a bright static prompt on overnight.

8. CPU and thermal validation

This step verifies the cooler installation, fan control, and general CPU stability.

Install monitoring tools

```
sudo apt install lm-sensors stress-ng
sudo sensors-detect
```

For `sensors-detect`:

- In most cases, answering the defaults is fine.
- It probes for sensor chips and configures the system to expose them.

Watch temperatures live

```
watch -n 1 sensors
```

What to look for:

- Idle temperatures should be plausible, not runaway.
- Under load, temperatures should rise and then settle.
- Fans should ramp smoothly.

Run a CPU stress test

Example:

```
stress-ng --cpu 16 --cpu-method matrixprod --timeout 30m --metrics-brief
```

What this does:

- Loads the CPU heavily for 30 minutes.
- Helps reveal thermal issues, unstable BIOS settings, or marginal cooling.

Interpretation

Good result:

- No crashes.
- No thermal shutdown.
- Temperatures stabilize rather than climbing uncontrollably.
- System remains responsive enough to observe.

Bad result:

- Instant crash or reboot suggests a serious hardware or power issue.
- Fast thermal runaway suggests bad cooler mounting, pump failure, or airflow problems.

Optional deeper validation

For additional confidence:

- Run a longer 2-hour CPU stress test.
- Repeat with the case panels fully installed, since airflow changes matter.

9. In-OS memory validation

Memtest86+ is great, but it does not fully replace in-OS testing.

Install memtester

```
sudo apt install memtester
```

How to run it

Do not allocate all system RAM. Leave some memory for the OS.

Example for a 64 GB system:

```
sudo memtester 48G 3
```

Meaning:

- `48G` asks memtester to exercise 48 GB of RAM.
- `3` means three passes.

What to expect

The test prints a series of memory patterns and whether each passed.

Good result:

- All tests pass with no reported failures.

Bad result:

- Any repeatable failures suggest memory instability, often due to EXPO, BIOS maturity, or a DIMM issue.

If memtester fails but Memtest86+ passed

That can happen. Mixed real-world OS activity stresses the system differently. If it fails:

1. Revert to stock memory speed.
2. Retest.
3. Update BIOS if you have not already.

4. Then re-enable EXPO cautiously.

10. NVMe and storage validation

This confirms that the Samsung 990 Pro is healthy and performs normally.

Health check with SMART/NVMe log

Install tools:

```
sudo apt install smartmontools nvme-cli
```

Run:

```
sudo smartctl -a /dev/nvme0n1
sudo nvme smart-log /dev/nvme0
```

What to look for:

- No critical warnings.
- Temperature is reasonable.
- Media and data integrity errors are zero.
- Available spare is healthy.

Performance and stress with fio

Install fio:

```
sudo apt install fio
```

Create a test directory on the NVMe drive and run:

```
fio --name=randrw \
    --filename=$HOME/fio-test-file \
    --size=20G \
    --bs=128k \
    --rw=randrw \
    --rwmixread=70 \
    --iodepth=32 \
    --numjobs=4 \
    --runtime=300 \
    --time_based \
    --group_reporting
```

What this does:

- Creates a 20 GB test file.

- Mixes reads and writes for 5 minutes.
- Exercises queueing and sustained activity.

Afterward, inspect logs:

```
sudo dmesg -l err,crit,alert,emerg
```

Good result:

- No I/O errors, resets, or NVMe timeouts.

Cleanup:

```
rm -f $HOME/fio-test-file
```

11. GPU stability and compute validation

Basic graphics validation

Install a lightweight benchmark or test app.

Useful checks:

- Run a 3D application.
- Watch for artifacts, crashes, or display blanking.
- Confirm GPU utilization moves in `nvidia-smi`.

CUDA/toolkit validation (optional)

If the workstation will be used for robotics, perception, or ML-adjacent experiments, CUDA validation is useful.

After installing the appropriate CUDA toolkit for Ubuntu 24.04, verify:

```
nvcc --version
nvidia-smi
```

Expected result:

- `nvcc` reports an installed toolkit version.
- `nvidia-smi` remains healthy and consistent with the installed driver.[cite:2]

Sustained GPU observation

Install a GPU monitor:

```
sudo apt install nvtop
nvtop
```

What to look for:

- GPU utilization tracks workload changes.
- VRAM usage changes appropriately.
- No sudden driver resets occur.

If the desktop freezes or the display driver resets under GPU load, check:

- Power connector seating.
- Driver version maturity.
- Journal logs: `journalctl -b | grep -Ei 'nvrm|nvidia|xid'`

12. USB, networking, and I/O validation

For robotics and embedded work, I/O reliability matters.

USB validation

Plug in a known USB 3 device such as a fast flash drive or camera.

Run:

```
lsusb -t
```

What to look for:

- The device should negotiate at the expected speed tier.

Practical validation:

- Copy a large file to/from a fast USB SSD.
- Watch for disconnects or repeated reconnect sounds/messages.
- Check logs with `dmesg | tail -n 100`.

Ethernet validation

Run:

```
ip a
ping -c 20 1.1.1.1
```

```
ping -c 20 ubuntu.com
```

What to look for:

- Stable packet flow.
- No random interface drops.

Wi-Fi validation

If you intend to use Wi-Fi:

```
nmcli device status
nmcli dev wifi list
```

Then connect and test sustained transfer or streaming.

13. Suspend/resume validation

Modern Linux systems can still fail in suspend/resume paths, especially with GPUs and multiple monitors.

How to test it

1. Save your work.
2. Suspend the machine:

```
systemctl suspend
```

3. Wake it after 1 to 5 minutes.
4. Repeat with:
    - One monitor.
    - Two monitors.
    - High refresh rate enabled.

What to validate after resume

- The desktop returns promptly.
- Both monitors come back correctly.
- Mouse and keyboard are responsive.
- Network reconnects.
- `nvidia-smi` still works.
- Logs stay clean.

Useful commands after resume:

```
nvidia-smi
sudo journalctl -b | tail -n 200
```

If resume fails only in the dual-4K/high-refresh configuration, treat that as a display stack stability issue rather than a core-platform failure.

14. Long-duration reliability test plan

Once individual subsystems pass, do a mixed soak test.

Recommended soak test

For 4 to 8 hours, combine:

- A CPU stress workload.
- Some disk activity.
- A video playback or light 3D workload.
- Normal desktop usage across both monitors.

Example approach:

1. Run `stress-ng` for CPU.
2. Run a shorter `fio` job occasionally.
3. Stream high-resolution video on one monitor.
4. Use the machine normally on the other.

Success criteria

- No crashes or lockups.
- No spontaneous reboots.
- No storage errors.
- No GPU Xid errors.
- Temperatures stay controlled.

Failure triage order

If something fails, back out changes in this order:

1. Disable EXPO.
2. Reduce display refresh rate.
3. Return to one monitor.
4. Re-seat GPU and power connectors.
5. Recheck BIOS version and driver version.

This isolates variables efficiently.

15. Recommended development baseline packages

After hardware validation, install a clean baseline for systems and robotics work:

```
sudo apt install \
  build-essential \
  git \
  curl \
  wget \
  vim \
  htop \
  tmux \
  tree \
  ripgrep \
  python3-pip \
  linux-headers-$(uname -r) \
  dkms \
  qemu-kvm \
  libvirt-daemon-system \
  virt-manager \
  mesa-utils \
  libdrm-dev \
  libgbm-dev
```

Validation for virtualization

```
lsmod | grep kvm
virsh list --all
```

Expected result:

- KVM modules are loaded.
- `virsh` works without a daemon or permission error after proper group setup.

This is especially useful if you want to experiment with kernels, distros, or toolchains without destabilizing the main install.

16. Stable long-term operating posture

For a workstation meant to stay reliable while still teaching useful skills:

- Keep one known-good BIOS profile saved.
- Avoid simultaneous changes to BIOS, kernel, and NVIDIA driver unless necessary.
- Change one subsystem at a time, then retest.
- Keep a simple bring-up log with date, BIOS version, kernel version, driver version, and notable observations.

- Once stable, create a backup image or snapshot strategy.

A very practical split is:

- **Stable baseline**: Ubuntu 24.04, conservative BIOS, validated EXPO or stock RAM, one proven NVIDIA driver branch.
- **Experimentation layer**: VMs, containers, alternate kernels, extra toolchains, and future display-stack experiments.

That lets the machine remain useful even while you explore riskier topics.

17. Quick validation checklist

Use this as the abbreviated acceptance list.

Hardware / firmware

- [ ] BIOS updated.
- [ ] CPU, RAM, SSD, GPU all detected.
- [ ] Idle BIOS temperatures reasonable.

Memory

- [ ] Memtest86+ passes at stock settings.
- [ ] Memtest86+ passes after EXPO is enabled.
- [ ] `memtester` passes in Ubuntu.

OS and storage

- [ ] Ubuntu installs cleanly.
- [ ] `journalctl -p err -b` contains no serious recurring errors.
- [ ] SMART/NVMe logs show no critical warnings.
- [ ] `fio` test completes without I/O errors.

GPU and displays

- [ ] `nvidia-smi` works.
- [ ] `glxinfo` shows NVIDIA renderer, not llvmpipe.
- [ ] One-monitor test stable.
- [ ] Two-monitor test stable.
- [ ] High refresh stable at chosen settings.

Reliability

- [ ] CPU stress test passes.
- [ ] Suspend/resume works.
- [ ] Mixed soak test passes.
- [ ] No unexplained crashes or reboots.

Sources

Ubuntu 24.04 users with RTX 5070-class GPUs have reported that out-of-box support may be incomplete until a current 57x-series or newer driver branch is installed.[cite:1][cite:2] Guidance for Secure Boot-enabled setups commonly involves MOK enrollment or module signing when installing modern NVIDIA drivers on Ubuntu 24.04.[cite:2]